

Tuyển Tập Bài Tập: Chặt Nhị Phân Song Song (Parallel Binary Search)

(16 Bài toán Tuyển chọn Phân cấp từ POI, SPOJ, AtCoder, Codeforces, VNOJ & Luogu)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)
Soạn thảo: Được hỗ trợ và soạn thảo bởi Agentic AI tool
Bản quyền © 2026 **Duc-Minh Vu**. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

Bảng Tổng Hợp 16 Bài Tập Tuyển Chọn

STT	Mã Bài / Tên Bài	Nguồn / Nền tảng	Cấp độ	Cấu trúc Dữ liệu Phối hợp	Độ phức tạp Mục tiêu
1	SPOJ METEORS (Meteors)	POI 2011 / SPOJ	Khởi động	Fenwick Tree (Range Add / Point Query)	$O((M + Q) \log M \log N)$
2	SPOJ MKTHNUM (K-th Number)	SPOJ	Khởi động	Fenwick Tree (Point Add / Range Sum)	$O((N + Q) \log N \log(\max A))$
3	SPOJ KQUERY (K-Query Offline)	SPOJ / VNOJ	Khởi động	Fenwick Tree trên mảng hoán vị	$O((N + Q) \log N)$
4	AtCoder AGC002D (Stamp Rally)	AtCoder Grand Contest	Trung cấp	DSU with Rollback (Graph Connectivity)	$O((M + Q) \log M \log N)$
5	Codeforces 1380E (Merging Towers)	Codeforces Edu 91	Trung cấp	DSU / Small-to-Large + PBS	$O((N + M) \log M)$
6	Luogu P3834 (Range K-th Smallest)	Luogu / Bản mẫu	Trung cấp	Fenwick Tree (Tối ưu RAM $O(N)$)	$O((N + Q) \log N \log V)$
7	Codeforces 482B (Interesting Array)	Codeforces Round 275	Trung cấp	Segment Tree / Prefix Bitwise AND	$O((N + Q) \log N)$
8	AtCoder ARC068E (Snuke Line)	AtCoder Regular Contest	Nâng cao	Fenwick Tree + Divisor Sweeping	$O(M\sqrt{N} + N \log M)$
9	Luogu P1527 (Matrix K-th Smallest)	国家集训队 / Luogu	Nâng cao	2D Fenwick Tree trên lưới tọa độ	$O((N^2 + Q) \log N \log V)$

STT	Mã Bài / Tên Bài	Nguồn / Nền tảng	Cấp độ	Cấu trúc Dữ liệu Phối hợp	Độ phức tạp Mục tiêu
10	Luogu P3332 / BZOJ 3110 (K-th Number Query)	ZJOI 2013 / Luogu	Nâng cao	Segment Tree với Lazy Tagging	$O((N + Q) \log^2 N)$
11	Codeforces 603E (Pastoral Oddities)	Codeforces Round 334	Nâng cao	Rollback DSU + Odd-component tracker	$O(M \log M \log N)$
12	Luogu P2617 (Dynamic Range K-th)	Luogu / Dynamic K-th	Nâng cao	CDQ Divide & Conquer / PBS trên thời gian	$O((N + Q) \log^2 N)$
13	Codeforces 1156E (Special Segments)	Codeforces Edu 64	Chỉnh phục	Cartesian Tree / PBS + Small-to-Large	$O(N \log^2 N)$
14	Luogu P3247 (HNOI 2016 Minimum LCM)	HNOI 2016 / Luogu	Chỉnh phục	Phân rã căn + Rollback DSU + PBS	$O((M + Q) \sqrt{M} \log N)$
15	Codeforces 1709E (XOR Tree Paths)	Codeforces Round 810	Chỉnh phục	Tree DSU / Small-to-Large Trie + PBS	$O(N \log^2 N)$
16	Luogu P3582 / POI 2015 Kinoman (Movie Goers)	POI 2015 / Luogu	Chỉnh phục	Segment Tree với Gmax + PBS	$O(N \log^2 N)$

1. Cấp Độ 1: Khởi Động & Nền Tảng (Warm-up & Fundamentals)

Bài 1: [SPOJ METEORS - Meteors \(POI 2011\)](#)

- **Nguồn:** Polish Olympiad in Informatics 2011 (POI XVIII) / [SPOJ METEORS](#) (Hoặc [Luogu P3527](#)).
- **Giới hạn:** $N, M, K \leq 3 \times 10^5$, Thời gian: 2.0s, Bộ nhớ: 128 MB.
- **Tóm tắt bài toán:** Quỹ đạo tròn gồm M trạm. N quốc gia, quốc gia i sở hữu một số trạm và cần tích lũy P_i thiên thạch. Có K đợt mưa, đợt thứ t tăng trạm trong cung tròn $[L_t, R_t]$ thêm A_t . Tìm đợt mưa sớm nhất mỗi quốc gia đạt chỉ tiêu.

Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện):** Lượng thiên thạch tích lũy tại mỗi quốc gia luôn tăng đơn điệu theo thời gian t . Các truy vấn độc lập nhưng cùng chia sẻ dòng thời gian K đợt mưa $\implies$ Áp dụng Parallel Binary Search trên $[1, K + 1]$.
- **Bậc 2 (Cấu trúc dữ liệu):** Cây Fenwick Tree kích thước M hỗ trợ Range Add và Point Query. Vì quỹ đạo là vòng tròn: nếu $L_t > R_t$, tách thành 2 đoạn cập nhật $[L_t, M]$ và $[1, R_t]$.
- **Bậc 3 (Chi tiết cài đặt & Chạy thử):**

- Dùng saturating addition: khi cộng dồn thiên thạch các trạm của quốc gia i , nếu `sum >= P[i]` thì dừng ngay để tránh tràn số `unsigned long long`.
- Hoàn tác cập nhật sau khi kiểm tra nhánh trái: gọi lại `range_add` với `-A[t]`. Không dùng `memset`.

Bài 2: [SPOJ MKTHNUM - K-th Number](#)

- **Nguồn:** [SPOJ MKTHNUM](#).
- **Giới hạn:** $N \leq 10^5$, $Q \leq 5 \times 10^4$, Thời gian: 1.5s, Bộ nhớ: 64 MB.
- **Tóm tắt bài toán:** Cho mảng A gồm N số nguyên. Mỗi truy vấn gồm bộ 3 (l, r, k) : tìm phần tử nhỏ thứ k trong đoạn $A[l \dots r]$.

Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện):** Thay vì dựng Persistent Segment Tree (tốn $O(N \log N)$ bộ nhớ, dễ vượt 64 MB), ta áp dụng PBS trên **miền giá trị** $[\min(A), \max(A)]$.
- **Bậc 2 (Cấu trúc dữ liệu):** Fenwick Tree kích thước N . Tại mỗi bước `mid`, các phần tử có $A[i] \leq \text{mid}$ được kích hoạt (đặt giá trị tại vị trí i thành 1).
- **Bậc 3 (Chi tiết cài đặt):**
 - Số phần tử $\leq \text{mid}$ trong $[l, r]$ là `cnt = bit.query(1, r)`.
 - Nếu $k \leq \text{cnt}$: đáp án nằm trong $[\text{left_val}, \text{mid}]$.
 - Nếu $k > \text{cnt}$: đáp án nằm trong $[\text{mid} + 1, \text{right_val}]$, cập nhật $k \leftarrow k - \text{cnt}$.
 - Thuật toán đạt $O((N + Q) \log N \log(\max A - \min A))$ với bộ nhớ chỉ $O(N)$ cực kỳ khiêm tốn.

Bài 3: [SPOJ KQUERY - K-Query Offline](#)

- **Nguồn:** [SPOJ KQUERY](#) / [VNOI KQUERY](#).
- **Giới hạn:** $N \leq 3 \times 10^4$, $Q \leq 2 \times 10^5$, Thời gian: 0.5s, Bộ nhớ: 64 MB.
- **Tóm tắt bài toán:** Cho dãy A gồm N phần tử. Mỗi truy vấn (i, j, k) đếm số phần tử trong $A[i \dots j]$ có giá trị lớn hơn k .

Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện):** Đây là bài toán tiền đề cho PBS. Có thể giải bằng Quét sự kiện (Sweep-line) sắp xếp giảm dần hoặc PBS trên trục giá trị.
- **Bậc 2 (Cấu trúc dữ liệu):** Fenwick Tree đếm tần số các vị trí đã kích hoạt.
- **Bậc 3 (Chi tiết cài đặt):** Rời rạc hóa giá trị. Đẩy các phần tử có $A[p] > k$ vào BIT. Kết quả là `query(j) - query(i - 1)`.

2. Cấp Độ 2: Trung Cấp & Tích Hợp Cấu Trúc Dữ Liệu (Intermediate)

Bài 4: [AtCoder AGC002D - Stamp Rally](#)

- **Nguồn:** [AtCoder Grand Contest 002 - Problem D](#).

- **Giới hạn:** $N, M, Q \leq 10^5$, Thời gian: 2.0s, Bộ nhớ: 256 MB.
- **Tóm tắt bài toán:** Cho đồ thị vô hướng N đỉnh, M cạnh. Mỗi truy vấn gồm (x, y, z) : hai người chơi bắt đầu tại đỉnh x và đỉnh y . Họ muốn ghé thăm tổng cộng ít nhất z đỉnh phân biệt. Tìm chỉ số cạnh lớn nhất nhỏ nhất (minimax edge index) trong hành trình của họ.

Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện):** Nếu chỉ số cạnh lớn nhất cho phép là mid , hai người chơi chỉ có thể di chuyển trong các thành phần liên thông tạo bởi các cạnh từ $1 \dots \text{mid}$. Tổng số đỉnh họ có thể ghé thăm tăng đơn điệu theo $\text{mid} \implies$ Áp dụng PBS trên dòng thời gian cạnh $t \in [1, M]$.
- **Bậc 2 (Cấu trúc dữ liệu):** DSU with Rollback (quản lý kích thước thành phần liên thông sz).
- **Bậc 3 (Chi tiết cài đặt):**
 - Tại bước mid : thêm các cạnh từ $L \dots \text{mid}$ vào DSU.
 - Với truy vấn (x, y, z) :
 - Nếu x và y cùng thành phần: tổng số đỉnh là $sz[\text{find}(x)]$.
 - Nếu khác thành phần: tổng số đỉnh là $sz[\text{find}(x)] + sz[\text{find}(y)]$.
 - Nếu tổng số đỉnh $\geq z$: đáp án $\leq \text{mid}$ (rẽ trái), ngược lại rẽ phải.
 - Hoàn tác các cạnh vừa thêm trước khi gọi đệ quy.

Bài 5: [Codeforces 1380E - Merging Towers](#)

- **Nguồn:** [Educational Codeforces Round 91 - Problem E](#).
- **Giới hạn:** $N, M \leq 2 \times 10^5$, Thời gian: 2.0s, Bộ nhớ: 512 MB.
- **Tóm tắt bài toán:** Có N đĩa ban đầu xếp trên M tháp. Thực hiện $M - 1$ thao tác gộp: gộp tháp B vào tháp A . Định nghĩa độ phức tạp sắp xếp là số cặp đĩa liên tiếp $(i, i + 1)$ nằm ở hai tháp khác nhau. In ra độ phức tạp này ban đầu và sau mỗi thao tác gộp.

Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện):** Có thể giải Online bằng DSU gộp tập hợp nhỏ vào lớn (Small-to-Large), hoặc giải Offline bằng PBS: với mỗi cặp $(i, i + 1)$, tìm thời điểm sớm nhất mà đĩa i và đĩa $i + 1$ nằm cùng một tháp.
- **Bậc 2 (Cấu trúc dữ liệu):** Sau khi tìm được thời điểm t_i cho mỗi cặp $(i, i + 1)$ thông qua PBS, mỗi thời điểm t sẽ làm giảm số phân đoạn đi 1.
- **Bậc 3 (Chi tiết cài đặt):** Khởi tạo đáp án ban đầu là $N - 1$. Tại mỗi sự kiện gộp t , trừ đi số cặp có $t_i = t$.

Bài 6: [Luogu P3834 - Range K-th Smallest](#)

- **Nguồn:** [Luogu P3834](#) (POJ 2761 / Bản mẫu).
- **Giới hạn:** $N, Q \leq 2 \times 10^5$, Thời gian: 1.0s, Bộ nhớ: 64 MB (Siêu hạn chế).
- **Tóm tắt bài toán:** Tìm phần tử nhỏ thứ k trong đoạn $[l, r]$. Yêu cầu không được ăn MLE bộ nhớ.

Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện):** Sử dụng khung code `OfflineRangeKthSolver` từ template chuẩn.
- **Bậc 2 (Cấu trúc dữ liệu):** Phân hoạch in-place danh sách phần tử và mảng truy vấn. Fenwick Tree kích thước N duy nhất.
- **Bậc 3 (Chi tiết cài đặt):**
 - Tách phần tử nhỏ hơn `mid` về nửa trái, lớn hơn về nửa phải.
 - Thao tác xóa BIT bằng cách gọi `bit.add(pos, -1)` đối với các phần tử vừa nạp. Tổng bộ nhớ tiêu thụ < 4 MB.

Bài 7: [Codeforces 482B - Interesting Array](#)

- **Nguồn:** [Codeforces Round 275 \(Div. 1\) - Problem B](#).
- **Giới hạn:** $N, M \leq 10^5$, Thời gian: 1.0s, Bộ nhớ: 256 MB.
- **Tóm tắt bài toán:** Cho M ràng buộc dạng: bitwise AND của đoạn $A[l_i \dots r_i]$ phải bằng q_i . Hãy dựng lại mảng A hoặc thông báo vô nghiệm.

Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện):** Các bit độc lập nhau (từ bit $0 \dots 29$). Với bit b , nếu q_i có bit $b = 1$, toàn bộ các phần tử trong $[l_i, r_i]$ bắt buộc phải có bit $b = 1$.
- **Bậc 2 (Cấu trúc dữ liệu):** Dùng mảng hiệu sai (Difference Array) để bật bit b trên đoạn $[l_i, r_i]$.
- **Bậc 3 (Chi tiết cài đặt):** Sau khi dựng mảng A , dùng Segment Tree / Sparse Table để kiểm tra lại toàn bộ M truy vấn xem đoạn $[l_i, r_i]$ có thực sự có AND bằng q_i không. Nếu không thỏa mãn, in "NO".

3. Cấp Độ 3: Nâng Cao & Đồ Thị Động (Advanced)

Bài 8: [AtCoder ARC068E - Snuke Line](#)

- **Nguồn:** [AtCoder Regular Contest 068 - Problem E](#).
- **Giới hạn:** $N \leq 3 \times 10^5$, $M \leq 10^5$, Thời gian: 2.0s, Bộ nhớ: 256 MB.
- **Tóm tắt bài toán:** Có N loại quà lưu niệm, loại i bán ở các ga từ l_i đến r_i . Với mỗi $d \in [1, M]$, một chuyến tàu dừng ở các ga là bội số của d ($d, 2d, 3d, \dots$). Tính số loại quà mà hành khách có thể mua được trên chuyến tàu d .

Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện):** Phân chia kích thước đoạn quà:
 - Nếu độ dài đoạn $r_i - l_i + 1 \geq d$: đoạn này chắc chắn chứa ít nhất một bội số của d . Ta luôn mua được!
 - Nếu độ dài đoạn $< d$: đoạn chỉ được mua nếu tồn tại bội số của d nằm gọn trong $[l_i, r_i]$.
- **Bậc 2 (Cấu trúc dữ liệu):** Sắp xếp các đoạn quà theo độ dài tăng dần. Duyệt d từ $1 \dots M$.
- **Bậc 3 (Chi tiết cài đặt):** Khi d tăng, các đoạn có độ dài $< d$ được nạp vào Fenwick Tree (cộng 1 vào l_i và trừ 1 vào $r_i + 1$). Với mỗi d , duyệt qua các bội số $d, 2d, \dots$ và lấy tổng điểm trên BIT.

Bài 9: [Luogu P1527 - Matrix K-th Smallest](#)

- **Nguồn:** [Chinese National Training Team / Luogu P1527](#).
- **Giới hạn:** Ma trận $N \times N$ với $N \leq 500$, $Q \leq 6 \times 10^4$, Thời gian: 2.5s, Bộ nhớ: 256 MB.
- **Tóm tắt bài toán:** Cho ma trận $N \times N$. Mỗi truy vấn gồm (x_1, y_1, x_2, y_2, k) : tìm phần tử nhỏ thứ k trong hình chữ nhật con có góc trên-trái (x_1, y_1) và góc dưới-phải (x_2, y_2) .

Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện):** PBS trên miền giá trị $[V_{\min}, V_{\max}]$ của các phần tử ma trận.
- **Bậc 2 (Cấu trúc dữ liệu):** Cây Fenwick Tree 2 chiều (2D Fenwick Tree) kích thước $N \times N$.
- **Bậc 3 (Chi tiết cài đặt):**
 - Tại bước mid, kích hoạt tất cả các ô (r, c) có giá trị $\leq \text{mid}$ vào 2D BIT: `bit.add(r, c, 1)`.
 - Truy vấn hình chữ nhật bằng công thức bao hàm loại trừ 2D: `query(x2, y2) - query(x1 - 1, y2) - query(x2, y1 - 1) + query(x1 - 1, y1 - 1)`.
 - Hoàn tác bằng cách `bit.add(r, c, -1)` đối với các ô vừa nạp.

Bài 10: [Luogu P3332 / BZOJ 3110 - K-th Number Query](#)

- **Nguồn:** [ZJOI 2013 / Luogu P3332](#).
- **Giới hạn:** $N, M \leq 50000$, Thời gian: 2.0s, Bộ nhớ: 256 MB.
- **Tóm tắt bài toán:** Có N tập hợp rỗng ban đầu. Có 2 loại thao tác:
 - Loại 1: Thêm số nguyên c vào mọi tập hợp từ a đến b .
 - Loại 2: Hỏi số lớn thứ k trong hợp của các tập hợp từ a đến b .

Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện):** Thao tác cập nhật theo đoạn và hỏi số lớn thứ k . Áp dụng PBS trên miền giá trị $[-N, N]$.
- **Bậc 2 (Cấu trúc dữ liệu):** Segment Tree hỗ trợ Range Add và Range Sum (kèm Lazy Tagging) để đếm số lượng số đã chen trong đoạn tập hợp $[a, b]$.
- **Bậc 3 (Chi tiết cài đặt):**
 - Lưu ý đây là tìm số **lớn** thứ k , nên ta kiểm tra nhánh phải trước hoặc quy đổi về nhỏ thứ K' .
 - Nếu tổng số phần tử lớn hơn mid trong $[a, b]$ lớn hơn hoặc bằng k , rẽ sang nhánh giá trị lớn hơn.

Bài 11: [Codeforces 603E - Pastoral Oddities](#)

- **Nguồn:** [Codeforces Round 334 \(Div. 1\) - Problem E](#).
- **Giới hạn:** $N \leq 10^5$, $M \leq 3 \times 10^5$, Thời gian: 4.0s, Bộ nhớ: 256 MB.
- **Tóm tắt bài toán:** Đồ thị ban đầu rỗng. Lần lượt thêm M cạnh có trọng số. Sau mỗi cạnh được thêm, tìm trọng số lớn nhất nhỏ nhất của tập cạnh con sao cho mọi đỉnh trong đồ thị đều có bậc lẻ (tất cả các thành phần liên thông đều có số đỉnh là số chẵn).

Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện):** Một đồ thị có tập cạnh con thỏa mãn bậc lẻ khi và chỉ khi **mọi thành phần liên thông của nó đều có kích thước chẵn** (tức là số thành phần liên thông có kích thước lẻ bằng 0).
- **Bậc 2 (Cấu trúc dữ liệu):** DSU with Rollback duy trì biến đếm `odd_components` (số thành phần có kích thước lẻ). Khi gộp 2 thành phần lẻ thành 1 thành phần chẵn, `odd_components -= 2`.
- **Bậc 3 (Chi tiết cài đặt):**
- Kết hợp Chia để trị thời gian và hai con trỏ / PBS: với mỗi nửa thời gian, tìm ngưỡng trọng số cạnh nhỏ nhất đưa `odd_components` về 0.

Bài 12: [Luogu P2617 - Dynamic Range K-th Smallest Element](#)

- **Nguồn:** [Luogu P2617](#) / ZOJ 2112.
- **Giới hạn:** $N, M \leq 10^5$, Thời gian: 2.0s, Bộ nhớ: 256 MB.
- **Tóm tắt bài toán:** Tìm phần tử nhỏ thứ k trên đoạn $[l, r]$, kèm theo thao tác thay đổi giá trị $A[p] = x$.

Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện):** Một phép sửa $A[p] = x$ tương đương với: xóa giá trị cũ $(p, A[p], -1)$ và thêm giá trị mới $(p, x, +1)$.
- **Bậc 2 (Cấu trúc dữ liệu):** Coi các thao tác truy vấn và cập nhật là các sự kiện có thứ tự thời gian. Áp dụng PBS kết hợp Chia để trị CDQ.
- **Bậc 3 (Chi tiết cài đặt):** Duy trì tính thứ tự thời gian của các phép gán trong từng bước phân hoạch của PBS.

4. Cấp Độ 4: Chinh Phục & Thử Thách Đỉnh Cao (Mastery)

Bài 13: [Codeforces 1156E - Special Segments of Permutation](#)

- **Nguồn:** [Educational Codeforces Round 64 - Problem E](#).
- **Giới hạn:** $N \leq 2 \times 10^5$, Thời gian: 2.0s, Bộ nhớ: 256 MB.
- **Tóm tắt bài toán:** Cho hoán vị P độ dài N . Đếm số cặp (l, r) sao cho $l < r$ và $P_l + P_r = \max(P_l, \dots, P_r)$.

Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện):** Dựng Cây Descartes (Cartesian Tree) trên mảng P theo giá trị lớn nhất. Mỗi nút u quản lý đoạn $[L_u, R_u]$ có P_u là cực đại.
- **Bậc 2 (Cấu trúc dữ liệu):** Cần tìm số cặp $l \in [L_u, u - 1]$ và $r \in [u + 1, R_u]$ thỏa $P_l + P_r = P_u$.
- **Bậc 3 (Chi tiết cài đặt):**
- Kỹ thuật Duyệt nhánh nhỏ (Heuristic Small-to-Large): Nếu nửa trái ngắn hơn nửa phải, duyệt qua mọi $l \in [L_u, u - 1]$, kiểm tra xem vị trí của giá trị $P_u - P_l$ có nằm trong $[u + 1, R_u]$ hay không (thông qua mảng nghịch đảo vị trí `pos`).
- Độ phức tạp: $O(N \log N)$ theo định lý phân rã kích thước.

Bài 14: [Luogu P3247 - Minimum LCM \(HNOI 2016\)](#)

- **Nguồn:** Hunan Olympiad in Informatics 2016 (HNOI 2016) / [Luogu P3247](#).
- **Giới hạn:** $N, M, Q \leq 10^5$, Thời gian: 4.0s, Bộ nhớ: 512 MB.
- **Tóm tắt bài toán:** Đồ thị gồm các cạnh có 2 trọng số (a_i, b_i) . Truy vấn (u, v, A, B) : tìm đường đi giữa u và v sao cho $\max(a) = A$ và $\max(b) = B$.

Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện):** Bài toán 2 chiều độc lập (a, b) . Phân rã căn (Sqrt Decomposition) trên trọng số a , sắp xếp truy vấn và các cạnh theo trọng số b .
- **Bậc 2 (Cấu trúc dữ liệu):** Rollback DSU lưu giá trị cực đại $\max(a)$ và $\max(b)$ trong từng thành phần liên thông.
- **Bậc 3 (Chi tiết cài đặt):** Trong mỗi khối căn, các cạnh có a thuộc khối được thêm tạm thời vào DSU bằng cơ chế rollback, các cạnh có a nhỏ hơn khối được thêm vĩnh viễn theo thứ tự b tăng dần.

Bài 15: [Codeforces 1709E - XOR Tree Paths](#)

- **Nguồn:** [Educational Codeforces Round 132 - Problem E](#).
- **Giới hạn:** $N \leq 2 \times 10^5$, Thời gian: 3.0s, Bộ nhớ: 256 MB.
- **Tóm tắt bài toán:** Cho cây N đỉnh, mỗi đỉnh có trọng số A_u . Thay đổi trọng số ít đỉnh nhất sao cho không có đường đi đơn nào có XOR trọng số các đỉnh bằng 0.

Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện):** Đặt X_u là XOR từ gốc đến u . Đường đi giữa u và v có XOR bằng 0 khi và chỉ khi $X_u \oplus X_v = A_{\text{LCA}(u,v)}$.
- **Bậc 2 (Cấu trúc dữ liệu):** DFS kết hợp Small-to-Large trên cây với `std::set` hoặc Trie lưu các giá trị X .
- **Bậc 3 (Chi tiết cài đặt):** Nếu phát hiện xung đột tại cây con của u , tham lam thay đổi giá trị A_u thành một số cực lớn (khác biệt) và xóa sạch toàn bộ tập X của cây con đó (chi phí +1).

Bài 16: [Luogu P3582 / POI 2015 Kinoman - Movie Goers](#)

- **Nguồn:** Polish Olympiad in Informatics 2015 (POI XXII) / [Luogu P3582](#).
- **Giới hạn:** $N, M \leq 10^6$, Thời gian: 3.0s, Bộ nhớ: 128 MB.
- **Tóm tắt bài toán:** Cho dãy phim. Xem một bộ phim lần đầu nhận W_f điểm, nhưng xem từ lần thứ 2 trở đi bị phạt mất điểm (hoặc nhận 0 điểm). Tìm khoảng thời gian $[l, r]$ để tổng điểm thu được là lớn nhất.

Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện):** Đổi trục: cố định điểm kết thúc r , cập nhật điểm đóng góp của các vị trí bắt đầu l .
- **Bậc 2 (Cấu trúc dữ liệu):** Segment Tree duy trì giá trị lớn nhất trên đoạn (Range Add / Range Max).
- **Bậc 3 (Chi tiết cài đặt):** Khi di chuyển r , một bộ phim f xuất hiện: nó cộng W_f vào khoảng $[\text{prev}[f] + 1, r]$, và trừ W_f vào $[\text{prev2}[f] + 1, \text{prev}[f]]$. Lấy giá trị lớn nhất trên toàn cây tại mỗi

bước r .



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.